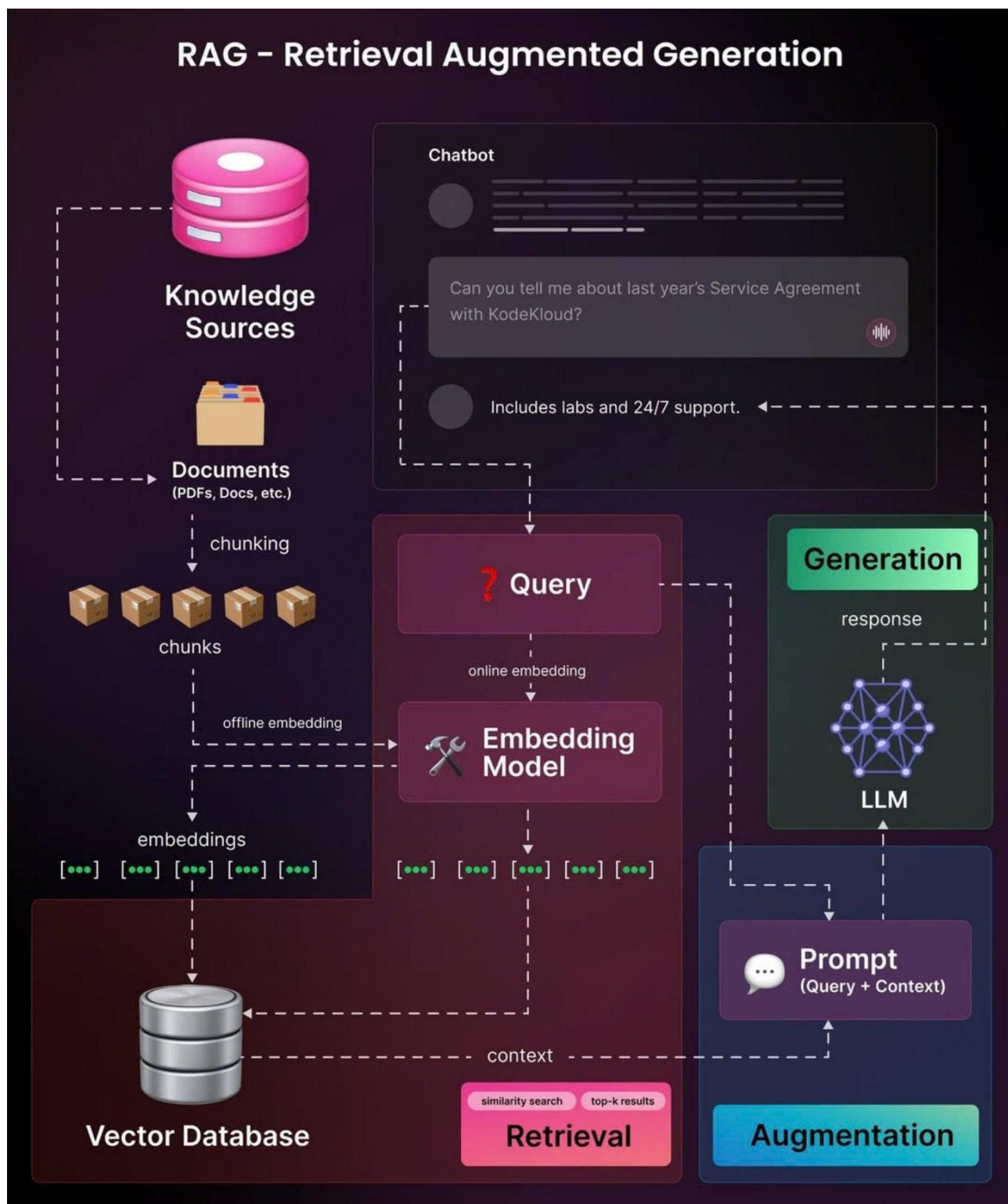


MỤC LỤC – HỆ THỐNG RAG

1	Kiến trúc RAG	1
2	Luồng công việc tổng thể (The High-Level Workflow)	2
3	Mô tả chi tiết các bước trong luồng thực hiện	3
3.1	Giai đoạn A: Luồng chuẩn bị dữ liệu: Xử lý dữ liệu (Data Ingestion Pipeline)	3
3.1.1	<i>Bước 1: Trích xuất và Phân tích cú pháp (Extraction and Parsing)</i>	3
3.1.1.1	Phân loại và Tiền xử lý luồng dữ liệu đầu vào (Ingestion and Routing)	3
3.1.1.2	Trích xuất nhận biết bố cục (Layout-Aware Parsing)	3
3.1.1.3	Làm sạch dữ liệu (Data Cleaning)	4
3.1.1.4	Gắn thẻ Siêu dữ liệu (Metadata Extraction and Tagging)	4
3.1.2	<i>Bước 2: Phân mảnh dữ liệu (Chunking)</i>	4
3.1.2.1	Các khái niệm cốt lõi	4
3.1.2.2	Các chiến lược Chunking	5
3.1.2.2.1	Kỹ thuật A: Phân mảnh đệ quy (Recursive Character Text Splitting)	5
3.1.2.2.2	Kỹ thuật B: Phân mảnh theo cấu trúc (Markdown / Structural Chunking)	5
3.1.2.2.3	Kỹ thuật C: Phân mảnh theo ngữ nghĩa - Nâng cao (Semantic Chunking)	5
3.1.2.3	Lời khuyên	5
3.1.3	<i>Bước 3: Nhúng Vector (Embedding)</i>	5
3.1.3.1	Nguyên lý hoạt động cốt lõi	6
3.1.3.2	Tiêu chí chọn Mô hình Nhúng (Embedding Model)	6
3.1.4	<i>Bước 4: Lập chỉ mục và Lưu trữ (Vector Indexing and Storage)</i>	6
3.1.4.1	Bản chất của Vector Database và Indexing	6
3.1.4.2	Kỹ thuật Lọc qua Siêu dữ liệu (Metadata Filtering)	7
3.2	Giai đoạn B: Luồng suy luận: Truy xuất và Sinh câu trả lời (Inference Pipeline)	7
3.2.1	<i>Bước 5: Tiền xử lý câu hỏi (Query Pre-processing)</i>	7
3.2.1.1	Ba Kỹ thuật Tiền xử lý	7
3.2.1.1.1	Kỹ thuật A: Viết lại câu hỏi độc lập (Query Rewriting)	7
3.2.1.1.2	Kỹ thuật B: Sinh câu hỏi phụ / Mở rộng truy vấn (Sub-Query Generation)	7
3.2.1.1.3	Kỹ thuật C: Tạo câu trả lời giả định (HyDE - Hypothetical Document Embeddings)	7
3.2.1.2	Lưu ý	8
3.2.2	<i>Bước 6: Truy xuất dữ liệu (Retrieval)</i>	8

3.2.2.1	Giải pháp kiến trúc	8
3.2.2.2	Kỹ thuật Reciprocal Rank Fusion (RRF)	8
3.2.2.3	Lời khuyên (Metadata Filtering)	9
3.2.3	<i>Bước 7: Xếp hạng lại (Re-ranking)</i>	9
3.2.3.1	Nguyên lý hoạt động: Bi-Encoder vs. Cross-Encoder	9
3.2.3.2	Lưu ý	9
3.2.4	<i>Bước 8: Xây dựng Prompt và Sinh văn bản (Prompt Construction and Generation)</i> ...	10
3.2.4.1	Nguyên lý Xây dựng Prompt cho RAG (Prompt Engineering)	10
3.2.4.1.1	Chỉ thị hệ thống (System Instruction):	10
3.2.4.1.2	Ngữ cảnh được cung cấp (Context)	10
3.2.4.1.3	Câu hỏi thực tế (User Query)	10
3.2.4.2	Đánh giá Thiết kế và Tối ưu hóa (Best Practices)	10
4	Phụ lục	11
4.1	<i>Building a robust RAG system</i>	11
4.2	<i>RAG – Moving parts</i>	12
4.3	<i>RAG developer’s stack</i>	13
4.4	<i>Open source RAG stack</i>	14
4.5	<i>Terms</i>	15

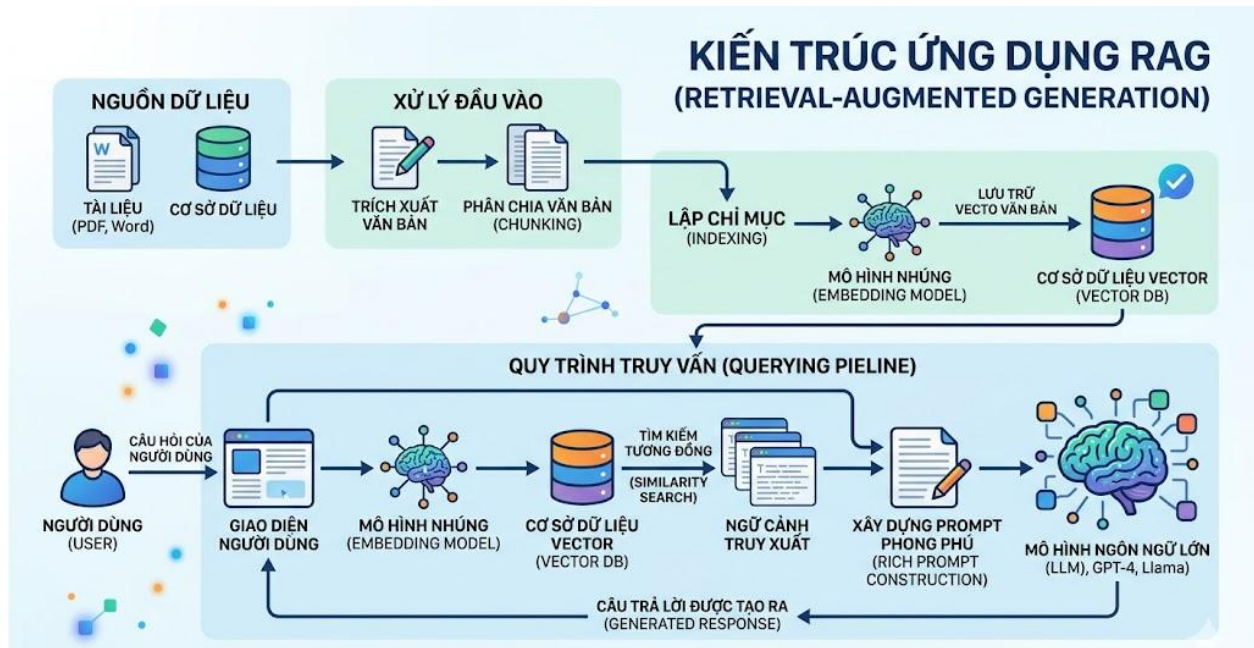
1 Kiến trúc RAG



Sơ đồ kiến trúc RAG bao gồm các thành phần chính như sau:

1. **Nguồn Dữ liệu (Source Data):** Nơi chứa dữ liệu thô như tài liệu PDF, Word hoặc cơ sở dữ liệu.
2. **Xử lý Đầu vào (Ingestion and Processing):** Dữ liệu được trích xuất, chia nhỏ (chunking) và chuyển đổi thành các vector (embeddings) để lưu trữ.

3. **Lập chỉ mục và Lưu trữ Vector (Indexing and Vector Storage):** Cơ sở dữ liệu Vector lưu trữ các đại diện toán học của dữ liệu để tìm kiếm nhanh chóng.
4. **Quy trình Truy vấn (Retrieval Pipeline):** Câu hỏi của người dùng được chuyển đổi thành vector và so sánh với cơ sở dữ liệu Vector để tìm ra thông tin liên quan nhất (Similarity Search).
5. **Tạo câu trả lời (Augmentation and Generation):** Thông tin truy xuất được kết hợp với câu hỏi ban đầu để tạo ra Prompt phong phú hơn, sau đó được gửi đến LLM để tạo ra câu trả lời cuối cùng.

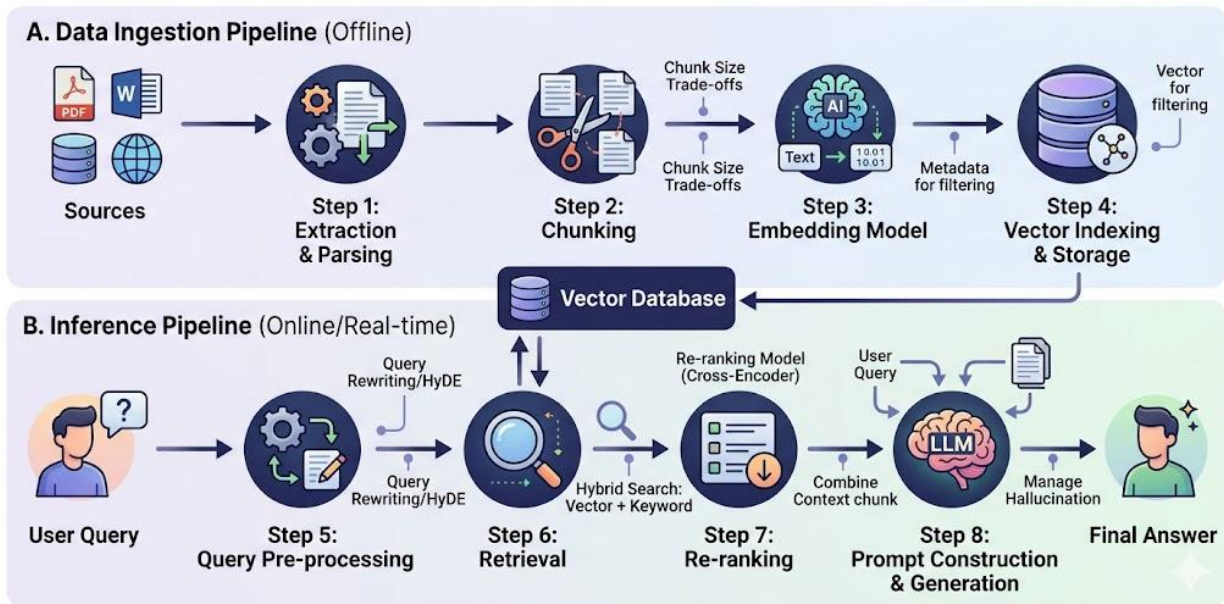


2 Luồng công việc tổng thể (The High-Level Workflow)

Kiến trúc của một ứng dụng RAG được chia thành hai luồng xử lý (pipeline) tách biệt về mặt vòng đời (lifecycle) nhưng gắn kết chặt chẽ về luồng dữ liệu:

- **Data Ingestion Pipeline (Luồng chuẩn bị dữ liệu - Offline):** Chịu trách nhiệm thu thập tài liệu thô, làm sạch, chia nhỏ, chuyển đổi thành các biểu diễn toán học (vector) và lưu trữ vào cơ sở dữ liệu. Luồng này chạy ngầm, theo lô (batch) hoặc kích hoạt khi có dữ liệu mới.
- **Inference Pipeline (Luồng suy luận - Online/Real-time):** Xảy ra khi người dùng đặt câu hỏi. Hệ thống sẽ xử lý câu hỏi, tìm kiếm ngữ cảnh liên quan nhất từ cơ sở dữ liệu đã chuẩn bị, và cung cấp cho Mô hình ngôn ngữ lớn (LLM) để tổng hợp câu trả lời cuối cùng.

RAG System Architecture: Ingestion & Inference Workflow



3 Mô tả chi tiết các bước trong luồng thực hiện

3.1 Giai đoạn A: Luồng chuẩn bị dữ liệu: Xử lý dữ liệu (Data Ingestion Pipeline)

3.1.1 Bước 1: Trích xuất và Phân tích cú pháp (Extraction and Parsing)

- Mô tả:** Thu thập dữ liệu từ nhiều nguồn (PDF, Word, Database, Web) và chuyển đổi thành văn bản thuần (plain text).
- Yếu tố kỹ thuật:** Bước này thường gặp thách thức lớn với các tài liệu có cấu trúc phức tạp. Ví dụ, khi xây dựng hệ thống tư vấn sâu bệnh cây trồng, các tài liệu chuyên ngành nông nghiệp thường chứa rất nhiều bảng biểu đặc tả loại thuốc, liều lượng và hình ảnh triệu chứng bệnh. Việc sử dụng các công cụ bóc tách (như OCR, PDF parser) cần giữ lại được cấu trúc logic thay vì chỉ trích xuất chữ thô.

3.1.1.1 Phân loại và Tiền xử lý luồng dữ liệu đầu vào (Ingestion and Routing)

Trước khi bóc tách, hệ thống cần biết nó đang đối mặt với loại file nào để điều hướng đến công cụ xử lý (parser) phù hợp.

Xác định định dạng MIME: Đừng chỉ dựa vào đuôi file (.pdf, .docx). Hãy dùng thư viện (như python-magic) để đọc header của file và xác định chính xác loại dữ liệu.

Đánh giá chất lượng (đặc biệt với PDF): Phân loại xem đó là PDF "chuẩn" (native PDF - có thể bôi đen copy chữ) hay là PDF dạng scan (chỉ là tập hợp các hình ảnh). Nếu là PDF scan, luồng dữ liệu bắt buộc phải đi qua OCR (Nhận dạng ký tự quang học).

3.1.1.2 Trích xuất nhận biết bố cục (Layout-Aware Parsing)

Đây là kỹ thuật quan trọng nhất trong RAG hiện đại. Thay vì gộp toàn bộ trang tài liệu thành một khối văn bản khổng lồ, hệ thống cần phân rã tài liệu thành các khối thông tin (elements) có phân loại rõ ràng.

Phân loại phần tử (Element Classification): Nhận diện đâu là Title (Tiêu đề bài), Header (Tiêu đề mục), Narrative Text (Đoạn văn nội dung), List Item (Mục liệt kê). Điều này giúp ích cực lớn cho bước Chunking (chia nhỏ dữ liệu) sau này.

Xử lý Bảng biểu (Table Extraction): Đây là "con ác mộng" của RAG. Ví dụ, khi xử lý các tài liệu đặc thù như cẩm nang nông nghiệp, một bảng hướng dẫn sử dụng thuốc bảo vệ thực vật nếu chỉ bóc tách thành văn bản thô sẽ bị vỡ nát cấu trúc hàng/cột, khiến LLM sau này đọc sai liều lượng hoặc

nhầm lẫn giữa các loại bệnh. Giải pháp là trích xuất bảng và giữ nguyên định dạng Markdown hoặc HTML để LLM dễ hiểu nhất.

Xử lý Hình ảnh (Image OCR and Captioning): Với các sơ đồ phức tạp, hình ảnh triệu chứng trên lá cây hoặc tài liệu quét mờ, bạn có thể tích hợp luồng xử lý riêng sử dụng các mô hình chuyên dụng như DeepSeek-OCR hoặc Vision LLM (như Llama 3.2 Vision) để đọc nội dung ảnh và chuyển thành mô tả văn bản (caption) trước khi lưu vào cơ sở dữ liệu.

3.1.1.3 Làm sạch dữ liệu (Data Cleaning)

Dữ liệu vừa bóc tách thường chứa rất nhiều “rác” làm nhiễu vector không gian và tốn token của LLM.

Loại bỏ thông tin lặp lại: Cắt bỏ Header/Footer của trang (ví dụ: tên sách, số trang lặp lại ở mọi trang).

Chuẩn hóa văn bản: Sửa lỗi dính chữ do lỗi font, chuẩn hóa Unicode (rất quan trọng với tiếng Việt), xóa các ký tự khoảng trắng thừa, ký tự ẩn.

3.1.1.4 Gắn thẻ Siêu dữ liệu (Metadata Extraction and Tagging)

Metadata là “chìa khóa vàng” để tăng độ chính xác khi tìm kiếm. Đi kèm với mỗi khối văn bản được bóc ra, bạn phải đính kèm các thông tin sau vào dạng JSON:

source: Tên file gốc hoặc URL.

page_number: Số trang (để sau này trích dẫn cho người dùng).

document_type: Loại tài liệu (ví dụ: Sách giáo trình, Báo cáo khoa học, Cẩm nang).

category/topic: Chủ đề chính (nếu phân loại được).

created_date: Ngày ban hành tài liệu (giúp ưu tiên dữ liệu mới hơn nếu có xung đột).

3.1.2 *Bước 2: Phân mảnh dữ liệu (Chunking)*

- **Mô tả:** Chia tài liệu dài thành các đoạn nhỏ (chunk) để phù hợp với giới hạn context window của LLM và tối ưu hóa việc tìm kiếm.
- **Yếu tố kỹ thuật:** Đây là bài toán về trade-off. Nếu chunk size quá nhỏ, bạn mất đi ngữ cảnh tổng thể (context). Nếu chunk size quá lớn, bạn đưa vào LLM nhiều thông tin nhiễu (noise) và tốn chi phí token. Các chiến lược phổ biến bao gồm *Fixed-size chunking*, *Sentence-based chunking*, hoặc *Semantic chunking* (chia theo ý nghĩa ngữ nghĩa).

Nếu ví hệ thống RAG như một người đọc sách, thì Parsing là việc mở sách ra, còn Chunking là việc xác định xem mỗi lần người đó nên đọc một câu, một đoạn, hay cả một trang để nhớ thông tin tốt nhất.

Việc cắt dữ liệu sai cách (ví dụ: cắt đứt đôi một công thức thuốc trừ sâu hay một triệu chứng bệnh) sẽ khiến ngữ cảnh bị phá vỡ hoàn toàn, dẫn đến việc LLM trả lời sai lệch (hallucination).

3.1.2.1 Các khái niệm cốt lõi

Trước khi viết mã, chúng ta cần thống nhất hai tham số định hình mọi chiến lược chunking:

- **Chunk Size (Kích thước phân mảnh):** Độ dài tối đa của một đoạn (đo bằng số ký tự hoặc số token). Kích thước lớn giữ được nhiều ngữ cảnh nhưng làm tăng nhiễu (noise) và tốn chi phí gọi LLM. Kích thước nhỏ giúp vector đối sánh chính xác hơn nhưng dễ mất ngữ cảnh tổng thể.
- **Chunk Overlap (Độ gối nhau):** Số lượng ký tự/token của đoạn trước được lặp lại ở đầu đoạn sau. **Đây là tham số bắt buộc phải có.**

- *Ví dụ thực tế:* Giả sử có câu: “Để trị bệnh đạo ôn trên lúa, nên sử dụng thuốc Regent 800WG”. Nếu không có overlap và vết cắt vô tình rơi vào giữa câu, đoạn 1 sẽ là “Để trị bệnh đạo ôn trên lúa, nên sử dụng”, đoạn 2 là “thuốc Regent 800WG”. Khi người dùng hỏi về bệnh đạo ôn, hệ thống tìm được đoạn 1 nhưng lại mất thông tin tên thuốc ở đoạn 2. Overlap giúp hàn gắn vết cắt này.

3.1.2.2 Các chiến lược Chunking

Dưới đây là 3 cấp độ phân mảnh từ cơ bản đến nâng cao. Để thực thi, bạn cần cài đặt thư viện: `pip install langchain-text-splitters`

3.1.2.2.1 Kỹ thuật A: Phân mảnh đệ quy (Recursive Character Text Splitting)

Đây là phương pháp “mặc định” và an toàn nhất cho hầu hết các loại văn bản. Nó sẽ cố gắng cắt văn bản dựa trên các ký tự tự nhiên (như `\n\n` đoạn văn, rồi đến `\n` câu, rồi đến dấu cách) để đảm bảo không cắt ngang giữa một từ hoặc một câu trừ khi bắt buộc.

3.1.2.2.2 Kỹ thuật B: Phân mảnh theo cấu trúc (Markdown / Structural Chunking)

Nếu ở Bước 1, bạn đã trích xuất dữ liệu và chuyển nó về định dạng Markdown (giữ được các thẻ `#` Tiêu đề 1, `##` Tiêu đề 2), thì phương pháp này cực kỳ mạnh mẽ. Nó nhóm thông tin dựa trên **logic cấu trúc của tài liệu** thay vì độ dài vật lý.

Lưu ý: Bạn sẽ thấy kết quả đầu ra tự động đính kèm metadata (ví dụ: `{'Header 1': 'Sổ tay quản lý dịch hại', 'Header 2': 'Cây ăn quả', 'Header 3': 'Bệnh thán thư trên xoài'}`). Điều này giúp bước tìm kiếm Vector sau này cực kỳ chính xác.

3.1.2.2.3 Kỹ thuật C: Phân mảnh theo ngữ nghĩa - Nâng cao (Semantic Chunking)

Thay vì đếm ký tự, chiến lược này sử dụng một mô hình Embedding nhỏ để đọc các câu, đo lường sự tương đồng về ý nghĩa toán học (Cosine Similarity) giữa chúng. Các câu có ý nghĩa giống nhau (cùng nói về một chủ đề) sẽ được gom vào một chunk. Khi chuyển sang ý khác, nó sẽ tự động cắt sang chunk mới.

Kỹ thuật này tốn tài nguyên tính toán hơn nhưng giải quyết triệt để bài toán “cắt đứt mạch văn”.

3.1.2.3 Lời khuyên

Trong các hệ thống tư vấn hỏi đáp chuyên môn có độ phức tạp cao, **không có một phương pháp chunking nào là hoàn hảo tuyệt đối**. Cách tốt nhất mà các kiến trúc sư phần mềm thường làm là áp dụng **Parent-Child Chunking** (hay còn gọi là Small-to-Big Retrieval):

- Cắt tài liệu thành các chunk rất nhỏ (Child - mức độ câu) để phục vụ việc đối sánh Vector thật chính xác.
- Tuy nhiên, khi tìm thấy Child, hệ thống không trả về Child đó cho LLM, mà trả về nguyên cả một đoạn lớn chứa Child đó (Parent - mức độ đoạn/mục) để LLM có đầy đủ bức tranh tổng thể.

3.1.3 Bước 3: Nhúng Vector (Embedding)

- **Mô tả:** Đưa các chunk văn bản qua một Embedding Model để chuyển đổi thành các vector số học chiều cao. Các đoạn văn bản có ý nghĩa tương đồng sẽ có vị trí vector gần nhau trong không gian.
- **Yếu tố kỹ thuật:** Lựa chọn mô hình embedding ảnh hưởng trực tiếp đến chất lượng tìm kiếm. Các mô hình chuyên biệt cho ngôn ngữ cụ thể hoặc domain cụ thể thường mang lại kết quả tốt hơn các mô hình đa dụng cơ bản.

Đây chính là "trái tim" của khả năng tìm kiếm ngữ nghĩa trong RAG.

Nếu không có bước này, hệ thống chỉ có thể tìm kiếm theo từ khóa thô (keyword matching). Embedding là phép thuật toán học giúp máy tính “hiểu” được ý nghĩa ẩn sau câu chữ.

3.1.3.1 Nguyên lý hoạt động cốt lõi

Embedding là quá trình đưa một đoạn văn bản (chunk) qua một mạng nơ-ron nhân tạo để chuyển đổi nó thành một mảng các con số (vector) trong không gian n-chiều.

Tại sao điều này lại quan trọng? Trong không gian toán học này, khoảng cách giữa các vector thể hiện mức độ tương đồng về mặt ý nghĩa. Ví dụ: Trong một hệ thống tư vấn sâu bệnh cây trồng, người nông dân có thể nhập câu hỏi về “cháy lá lúa”. Văn bản tài liệu gốc của bạn lại dùng từ học thuật là “bệnh đạo ôn”. Tìm kiếm từ khóa truyền thống sẽ thất bại hoàn toàn. Nhưng với Embedding, mô hình hiểu rằng “cháy lá lúa” và “đạo ôn” có chung ngữ cảnh không gian mạng, vector của chúng sẽ nằm rất gần nhau, giúp hệ thống truy xuất chính xác tài liệu cần thiết.

3.1.3.2 Tiêu chí chọn Mô hình Nhúng (Embedding Model)

Không phải mô hình nào cũng phù hợp cho mọi bài toán. Khi lựa chọn, bạn cần cân nhắc 3 yếu tố:

- **Hỗ trợ ngôn ngữ:** Với tài liệu tiếng Việt, tuyệt đối không dùng các mô hình chỉ chuyên tiếng Anh. Bạn cần các mô hình đa ngữ (Multilingual) có hiệu năng cao.
- **Kích thước Vector (Dimensions):** Vector càng dài (ví dụ: 1536 chiều của OpenAI, 1024 chiều của BAAI/bge-m3) thì càng chứa nhiều thông tin ngữ nghĩa, nhưng lại tốn dung lượng RAM/Ổ cứng và thời gian tính toán. Các mô hình nhỏ (384 chiều như MiniLM) chạy nhanh nhưng độ chính xác kém hơn một chút.
- **Môi trường triển khai:** Trong quá trình nghiên cứu và phát triển trên môi trường Windows 11 cùng VS Code, việc tải các mô hình mã nguồn mở chạy cục bộ (Local) là lựa chọn tối ưu nhất để tránh chi phí gọi API và không bị giới hạn giới hạn tốc độ (rate limit).

3.1.4 Bước 4: Lập chỉ mục và Lưu trữ (Vector Indexing and Storage)

- **Mô tả:** Lưu trữ các vector cùng với metadata (nguồn tài liệu, số trang, tiêu đề) vào Vector Database.
- **Yếu tố kỹ thuật:** Metadata đóng vai trò cực kỳ quan trọng cho việc lọc (filtering) trước hoặc sau khi tìm kiếm vector, giúp thu hẹp phạm vi tìm kiếm và tăng độ chính xác.

Bước 4 chính là việc xây dựng một “nhà kho” để lưu trữ và sắp xếp các dải số này sao cho việc tìm kiếm sau này diễn ra trong chớp mắt.

3.1.4.1 Bản chất của Vector Database và Indexing

Nếu bạn lưu 1 triệu vector vào một mảng (array) thông thường, mỗi khi có câu hỏi mới, hệ thống sẽ phải quét tuần tự từ đầu đến cuối để đo khoảng cách (độ phức tạp $O(N)$). Điều này cực kỳ chậm.

Vector Database (như ChromaDB, Qdrant, Milvus, Qdrant) giải quyết bài toán này bằng **Thuật toán lập chỉ mục (Indexing Algorithm)**, phổ biến nhất là **HNSW (Hierarchical Navigable Small World)**.

- Thay vì quét toàn bộ, HNSW xây dựng một đồ thị đa tầng. Nó cho phép nhảy vọt qua các không gian vector không liên quan và nhanh chóng hội tụ về các vector gần giống với câu hỏi nhất ở tốc độ $O(\log N)$.

3.1.4.2 Kỹ thuật Lọc qua Siêu dữ liệu (Metadata Filtering)

Chỉ tìm kiếm bằng Vector đôi khi không đủ chính xác. Ví dụ: Người dùng hỏi “Cách trị rầy nâu”, nhưng họ chỉ muốn áp dụng cho “cây lúa” chứ không phải “cây ăn quả”.

Để giải quyết, khi lưu trữ vector, chúng ta phải đính kèm Metadata. Ở bước tìm kiếm (Step 6), chúng ta sẽ dùng bộ lọc (Filter) để giới hạn không gian tìm kiếm trước (Pre-filtering) hoặc sau (Post-filtering) khi chạy nội suy vector.

3.2 *Giai đoạn B: Luồng suy luận: Truy xuất và Sinh câu trả lời (Inference Pipeline)*

3.2.1 *Bước 5: Tiền xử lý câu hỏi (Query Pre-processing)*

- **Mô tả:** Biến đổi câu hỏi gốc của người dùng thành một định dạng dễ truy xuất hơn.
- **Yếu tố kỹ thuật:** Người dùng thường đặt câu hỏi ngắn gọn, thiếu chủ ngữ hoặc chứa từ viết tắt. Các kỹ thuật như *Query Rewriting* (dùng một LLM nhỏ để viết lại câu hỏi rõ ràng hơn) hoặc *HyDE* (*Hypothetical Document Embeddings*) thường được áp dụng ở đây để cải thiện đối sánh ngữ nghĩa.

Khi người dùng nhập một câu hỏi vào hệ thống RAG, họ thường viết theo ngôn ngữ tự nhiên, rất ngắn gọn, đôi khi mơ hồ, sai chính tả, hoặc phụ thuộc nặng nề vào ngữ cảnh của các câu chuyện trước đó (ví dụ: “Thuốc này dùng thế nào?” khi câu trước vừa nhắc đến một loại thuốc cụ thể).

Nếu lấy trực tiếp câu hỏi thô này mang đi nhúng vector (Embedding) ở Bước 3, khoảng cách vector sẽ bị lệch và quá trình truy xuất (Retrieval) ở Bước 6 sẽ trả về các kết quả rất kém chất lượng.

Bước 5: Query Pre-processing (Tiền xử lý câu hỏi) giúp giải quyết bài toán này. Nó đóng vai trò chuyển đổi câu hỏi tự nhiên của người dùng thành một hoặc nhiều “vấn bản truy vấn” tối ưu nhất cho không gian vector.

3.2.1.1 Ba Kỹ thuật Tiền xử lý

3.2.1.1.1 Kỹ thuật A: Viết lại câu hỏi độc lập (Query Rewriting)

Kỹ thuật này cực kỳ quan trọng cho các hệ thống Chatbot RAG có lưu lịch sử trò chuyện (Conversation History). Nó dùng một LLM nhỏ để tổng hợp câu hỏi hiện tại và lịch sử chat thành một câu hỏi duy nhất, đầy đủ ngữ nghĩa và hoàn toàn độc lập (Standalone Query).

3.2.1.1.2 Kỹ thuật B: Sinh câu hỏi phụ / Mở rộng truy vấn (Sub-Query Generation)

Khi người dùng đặt một câu hỏi quá rộng (ví dụ: “Cách xử lý khi lúa bị dịch hại”), một vector đơn lẻ không thể bao phủ hết mọi khía cạnh thông tin trong cơ sở dữ liệu.

Kỹ thuật này dùng LLM để nhỏ câu hỏi lớn thành 3-4 câu hỏi cụ thể hơn ở các góc độ khác nhau. Hệ thống sẽ mang cả 4 câu hỏi này đi tìm kiếm vector và gộp kết quả lại (Kỹ thuật Multi-Query Retrieval).

3.2.1.1.3 Kỹ thuật C: Tạo câu trả lời giả định (HyDE - Hypothetical Document Embeddings)

Đây là một kỹ thuật rất độc đáo trong RAG nâng cao. Khoảng cách vector giữa một *Câu hỏi* và một *Đoạn văn bản câu trả lời* thường khá xa nhau trong không gian toán học (vì cấu trúc ngữ pháp khác nhau). Nhưng khoảng cách giữa một *Đoạn văn bản câu trả lời* và một *Đoạn văn bản câu trả lời khác* lại rất gần nhau.

HyDE sẽ dùng LLM viết một câu trả lời “giả định” (dù có thể chứa thông tin chưa chính xác hoàn toàn). Sau đó, hệ thống lấy chính **vector của câu trả lời giả định này** để đi tìm kiếm trong Vector DB. Kết quả tìm kiếm thực tế thường khớp ngữ nghĩa tốt hơn rất nhiều.

3.2.1.2 Lưu ý

Cần cân nhắc kỹ lực lượng cân bằng khi áp dụng Bước 5:

- **Độ chính xác (Accuracy) vs Độ trễ (Latency):** Việc gọi thêm LLM ở bước tiền xử lý câu hỏi giúp tăng độ chính xác của tài liệu lấy về lên rất nhiều. Tuy nhiên, nó sẽ cộng thêm thời gian vào tổng thời gian phản hồi (Inference Latency) của người dùng vì hệ thống phải chờ LLM sinh chữ trước khi tìm kiếm vector.
- **Giải pháp tối ưu:** Với các câu hỏi đơn lẻ và không có lịch sử chat, bạn có thể bỏ qua bước này để tối ưu tốc độ. Với luồng trò chuyện liên tục (Multi-turn Chatbot), việc áp dụng **Kỹ thuật A (Query Rewriting)** là bắt buộc để hệ thống không bị “mất trí nhớ”.

3.2.2 Bước 6: Truy xuất dữ liệu (Retrieval)

- **Mô tả:** Hệ thống sẽ chuyển câu hỏi đã xử lý thành vector (dùng chung Embedding Model ở Bước 3) và so sánh với Vector DB để lấy ra Top-K chunk có độ tương đồng cao nhất.
- **Yếu tố kỹ thuật:** Để tránh việc chỉ dựa vào Semantic Search (tìm theo ý nghĩa) đôi khi bỏ sót các từ khóa chính xác, xu hướng kiến trúc hiện đại thường sử dụng **Hybrid Search** — kết hợp giữa Vector Search và Keyword Search (như thuật toán BM25).

Đây là lúc hệ thống trích xuất **Top-K** đoạn văn bản (chunks) liên quan nhất từ Vector Database (đã xây dựng ở Bước 4) để chuẩn bị mớm cho LLM.

Tuy nhiên, nếu chỉ dùng một phương pháp tìm kiếm duy nhất, hệ thống RAG thường xuyên gặp lỗi “mù chữ” hoặc “ảo giác ngữ nghĩa”.

3.2.2.1 Giải pháp kiến trúc

Vấn đề cốt lõi: Cuộc chiến giữa “Ngữ nghĩa” và “Từ khóa”

Trong hệ thống RAG chuyên ngành (như quản lý dịch hại nông nghiệp), bạn sẽ đối mặt với hai thái cực tìm kiếm:

- **Dense Retrieval (Tìm kiếm Vector/Ngữ nghĩa):** Rất giỏi hiểu ý. Nếu người dùng hỏi “*cháy lá*”, nó biết tìm đoạn văn chứa chữ “*đạo ôn*”. **Nhược điểm:** Cực kỳ kém trong việc tìm chính xác các danh từ riêng, mã số, hoặc tên thuốc hóa học (ví dụ: “*Isoprothiolane 40EC*”).
- **Sparse Retrieval (Tìm kiếm Từ khóa/BM25):** Hoạt động như Google thời kỳ đầu. Rất giỏi tìm chính xác các từ khóa hoặc tên thuốc lạ. **Nhược điểm:** “Mù” ngữ nghĩa. Nếu hỏi “*thuốc trị nấm*”, nó sẽ bỏ qua tài liệu ghi là “*chất diệt khuẩn*”.

=> **Giải pháp kiến trúc: Hybrid Search (Tìm kiếm Lai)** kết hợp sức mạnh của cả hai.

3.2.2.2 Kỹ thuật Reciprocal Rank Fusion (RRF)

Khi bạn chạy song song hai luồng tìm kiếm (BM25 và Vector), mỗi luồng sẽ trả về một bảng xếp hạng (Rank) khác nhau. Làm sao để gộp chúng lại?

Thuật toán phổ biến nhất là **RRF (Dung hợp thứ hạng nghịch đảo)**.

Công thức tính điểm RRF cho một tài liệu:

$$RRF_{Score} = \frac{1}{k + Rank_{Vector}} + \frac{1}{k + Rank_{BM25}}$$

(Trong đó k thường là hằng số bằng 60 để chống nhiễu).

3.2.2.3 Lời khuyên (Metadata Filtering)

Dù Hybrid Search rất mạnh, nhưng để tối ưu tốc độ và độ chính xác (ví dụ: đảm bảo chỉ lấy tài liệu của “năm 2024” hoặc cây “lúa”), bạn nên áp dụng thêm Pre-filtering (Lọc siêu dữ liệu trước khi tìm kiếm vector). Trong ChromaDB hoặc LangChain, bạn có thể truyền thêm đối số `filter={"crop": "lúa"}` vào hàm tìm kiếm. Điều này sẽ giới hạn không gian đồ thị HNSW, giúp tìm kiếm nhanh hơn hẳn.

3.2.3 Bước 7: Xếp hạng lại (Re-ranking)

- **Mô tả:** Top-K kết quả lấy từ Vector DB thường chứa nhiều nhiễu. Bước này sử dụng một mô hình chuyên dụng (Cross-Encoder) để chấm điểm lại mức độ liên quan thực sự giữa câu hỏi và từng chunk ngữ cảnh, sau đó sắp xếp lại và chỉ giữ lại những chunk chất lượng nhất.
- **Yếu tố kỹ thuật:** Re-ranking tăng độ chính xác đáng kể nhưng đi kèm với trade-off về độ trễ (latency) vì việc tính toán tốn nhiều tài nguyên hơn.

Ở Bước 6, hệ thống đã dùng Hybrid Search để “vớt” lên được một tập hợp các tài liệu tiềm năng nhất. Tuy nhiên, dù thuật toán Hybrid tốt đến đâu, nó vẫn mắc phải một điểm yếu của kiến trúc Vector: **Thiếu sự chú ý chéo (Cross-Attention)**.

Nếu bạn nhồi nhét toàn bộ 10-20 kết quả từ Bước 6 vào LLM, mô hình ngôn ngữ sẽ bị “ngộp” thông tin, dẫn đến hiện tượng *Lost in the Middle* (LLM quên mất thông tin nằm ở giữa đoạn văn bản dài). **Bước 7: Re-ranking (Xếp hạng lại)** chính là “chiếc sàng lọc” cuối cùng để chọn ra một vài kết quả xuất sắc nhất, loại bỏ hoàn toàn nhiễu trước khi đưa cho LLM.

3.2.3.1 Nguyên lý hoạt động: Bi-Encoder vs. Cross-Encoder

Để hiểu tại sao cần Bước 7, bạn cần nắm rõ sự khác biệt giữa hai loại mô hình máy học:

- **Bi-Encoder (Sử dụng ở Bước 3 và Bước 6):** Mô hình này nhúng Câu hỏi và Tài liệu *hoàn toàn độc lập* thành hai vector, sau đó tính khoảng cách giữa chúng. Tốc độ cực nhanh (phù hợp quét hàng triệu bản ghi), nhưng không thể hiểu được mối quan hệ từ vựng phức tạp giữa câu hỏi và tài liệu.
- **Cross-Encoder (Sử dụng ở Bước 7):** Mô hình này nhận *cùng lúc* cả Câu hỏi và Tài liệu, ghép chúng lại với nhau và cho các từ vựng "nhìn" thấy nhau (Self-Attention) để chấm điểm tương đồng. Tốc độ rất chậm, nhưng độ chính xác ngữ nghĩa cực kỳ cao.

Quy trình chuẩn: Mở rộng phễu tìm kiếm ở Bước 6 (ví dụ: lấy Top 20) -> Đưa Top 20 qua Cross-Encoder ở Bước 7 -> Chỉ giữ lại Top 3 đến Top 5 để đưa vào Bước 8.

3.2.3.2 Lưu ý

Đối với hệ thống hỗ trợ tiếng Việt, bạn không thể dùng các mô hình Re-ranker chỉ chuyên tiếng Anh (như ms-marco).

Khi đưa Bước 7 vào hệ thống, bạn cần chấp nhận một sự đánh đổi cực lớn về **Độ trễ**:

- Cross-Encoder cực kỳ nặng nề. Nếu bạn đưa 100 kết quả vào hàm `predict()`, hệ thống có thể mất vài giây chỉ để chấm điểm (đặc biệt khi chạy trên CPU).
- Chỉ thiết lập Hybrid Search ở Bước 6 trả về tối đa **X** kết quả (ví dụ: **15-25 kết quả**). Không bao giờ đưa nhiều hơn số lượng này vào Cross-Encoder. Sau khi Re-ranking, hãy mạnh tay cắt bỏ (ví dụ: chỉ đưa **3-5 kết quả** xuất sắc nhất vào Prompt của LLM) để tiết kiệm Token và tránh làm LLM phân tâm.

3.2.4 Bước 8: Xây dựng Prompt và Sinh văn bản (Prompt Construction and Generation)

- **Mô tả:** Lắp ráp câu hỏi gốc của người dùng cùng với các ngữ cảnh (chunks) đã được tinh lọc ở Bước 7 vào một Prompt Template. LLM sẽ đọc khối thông tin này và tổng hợp thành câu trả lời cuối cùng.
- **Yếu tố kỹ thuật:** Prompt cần thiết lập ranh giới rõ ràng (ví dụ: "*Chỉ sử dụng các thông tin được cung cấp dưới đây để trả lời. Nếu không biết, hãy nói là không có thông tin*"). Điều này giúp quản lý tính toàn vẹn của dữ liệu và hạn chế tối đa tình trạng ảo giác (hallucination) của LLM.

Bước 8: Prompt Construction and Generation (Xây dựng Prompt và Sinh văn bản) là nghệ thuật “đóng gói” những khối dữ liệu này cùng với câu hỏi của người dùng, kèm theo các chỉ thị nghiêm ngặt để ép Mô hình ngôn ngữ lớn (LLM) trả lời một cách chính xác, an toàn và không bịa chuyện (hallucination).

Dưới đây là hướng dẫn kỹ thuật chi tiết để triển khai bước quan trọng này.

3.2.4.1 Nguyên lý Xây dựng Prompt cho RAG (Prompt Engineering)

Trong RAG, Prompt không chỉ đơn giản là đưa câu hỏi cho LLM. Một Prompt chuẩn kỹ thuật phải bao gồm 3 thành phần cốt lõi được phân tách rõ ràng:

3.2.4.1.1 Chỉ thị hệ thống (System Instruction):

Xác định vai trò của LLM và đưa ra bộ quy tắc cấm kỵ (Ví dụ: “Chỉ dựa vào ngữ cảnh”, “Không dùng kiến thức tự có”).

3.2.4.1.2 Ngữ cảnh được cung cấp (Context)

Chuỗi văn bản được gộp lại từ Top-K kết quả đã được tinh lọc ở Bước 7. Cần sử dụng các ký tự phân tách (như --- hoặc thẻ XML <context>) để LLM phân biệt rõ đâu là dữ liệu, đâu là câu lệnh.

3.2.4.1.3 Câu hỏi thực tế (User Query)

Câu hỏi nguyên bản của người dùng.

Nguyên tắc "Bức tường lửa" chống ảo giác: Bạn bắt buộc phải lập trình cho LLM một “lối thoát” an toàn. Nếu các tài liệu cung cấp hoàn toàn không chứa câu trả lời, LLM phải được lệnh nói như: “*Dựa trên tài liệu cung cấp, tôi không có thông tin...*” thay vì cố gắng tự đoán.

3.2.4.2 Đánh giá Thiết kế và Tối ưu hóa (Best Practices)

Để hệ thống RAG thực sự hoạt động hiệu quả ở môi trường thực tế (production), bạn cần lưu ý các kỹ thuật nâng cao sau ở Bước 8:

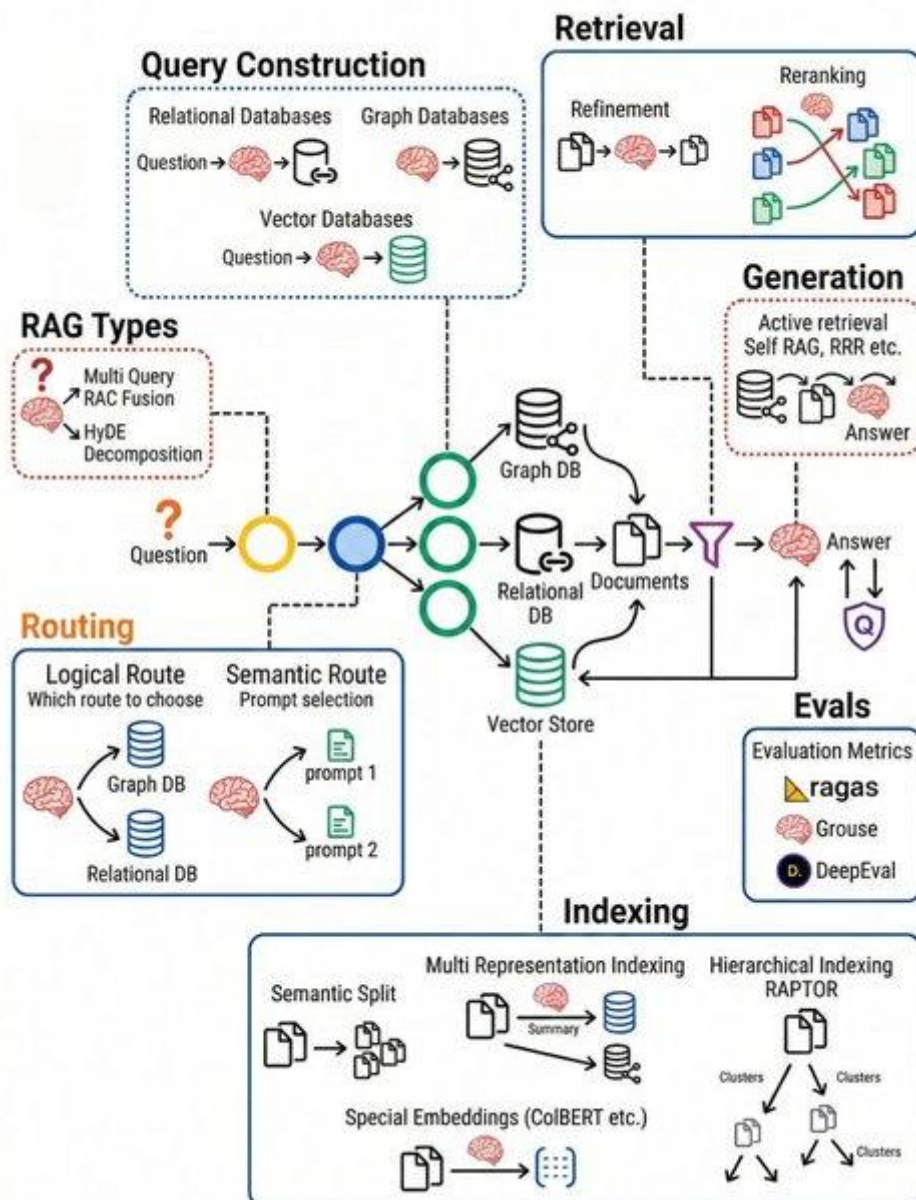
- **Kiểm soát Độ sáng tạo (Temperature = 0):** Tham số temperature của LLM xác định mức độ ngẫu nhiên của các từ được sinh ra. Đối với RAG chuyên ngành (đòi hỏi tính xác thực), luôn thiết lập temperature tiệm cận 0 để LLM hoạt động như một cỗ máy trích xuất thông tin logic thay vì một nhà văn sáng tạo.
- **Theo dõi Trích dẫn (Citation Tracking):** Để tăng độ tin cậy, bạn có thể yêu cầu LLM trích dẫn ngược lại tài liệu. Sửa lệnh trong Prompt thành: “*Mỗi khi bạn đưa ra một thông tin, hãy ghi chú nguồn bằng cách thêm [Tài liệu X] ở cuối câu.*” Hệ thống giao diện (UI) sau đó có thể bắt các thẻ này để hiển thị link cho người dùng đối chiếu.

- **Quản lý Context Window (Giới hạn Token):** Các LLM thường có giới hạn đầu vào (ví dụ: 4096 hoặc 8192 token). Hãy tính toán cẩn thận: độ dài của System Prompt + độ dài của Top_K kết quả (Bước 7) + độ dài User Query không bao giờ được vượt quá giới hạn này, nếu không LLM sẽ báo lỗi hoặc tự động cắt bớt ngữ cảnh. Lời khuyên là thiết lập một hàm đếm token (như sử dụng thư viện tiktoken) trước khi đóng gói Prompt.

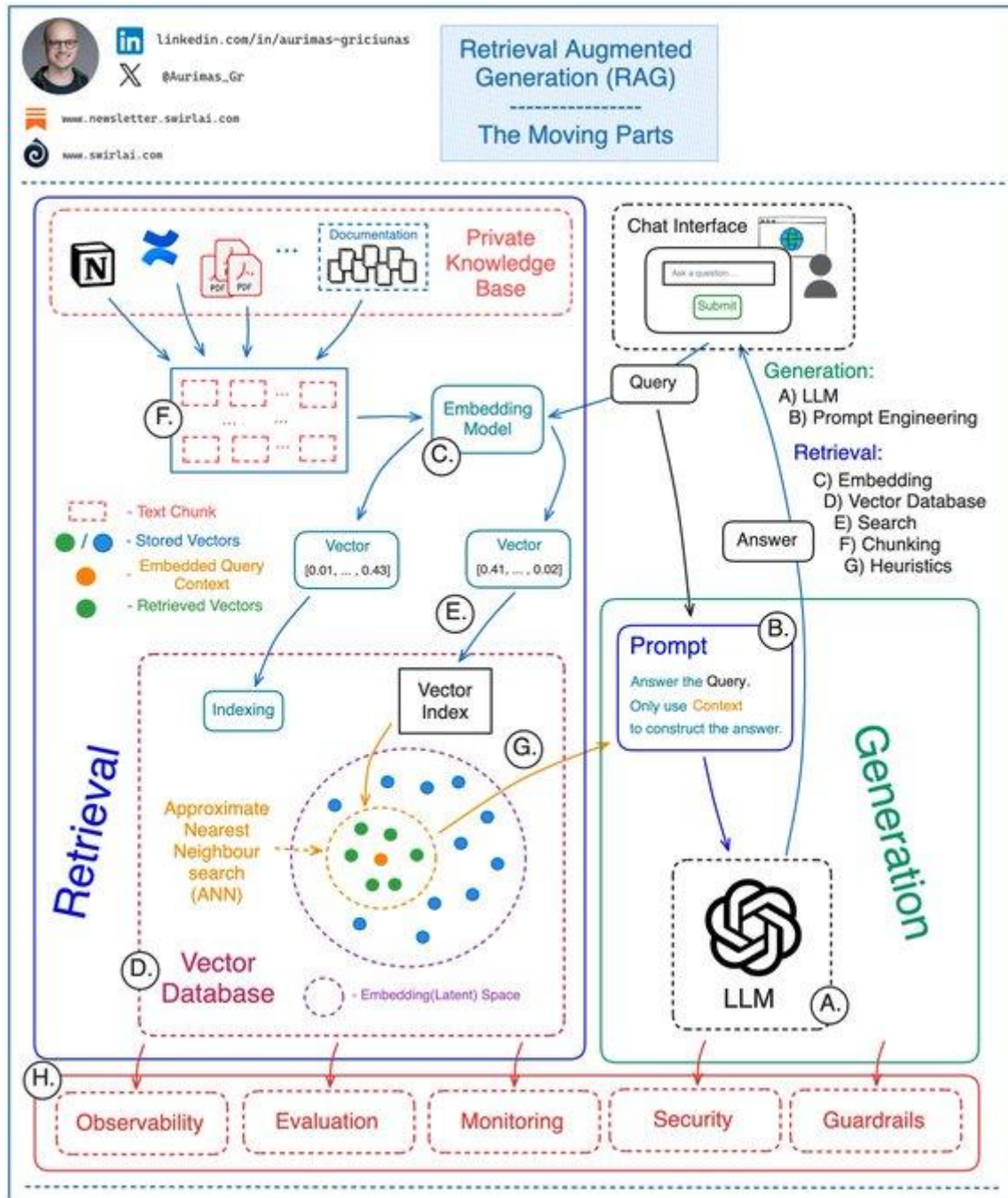
4 Phụ lục

4.1 Building a robust RAG system

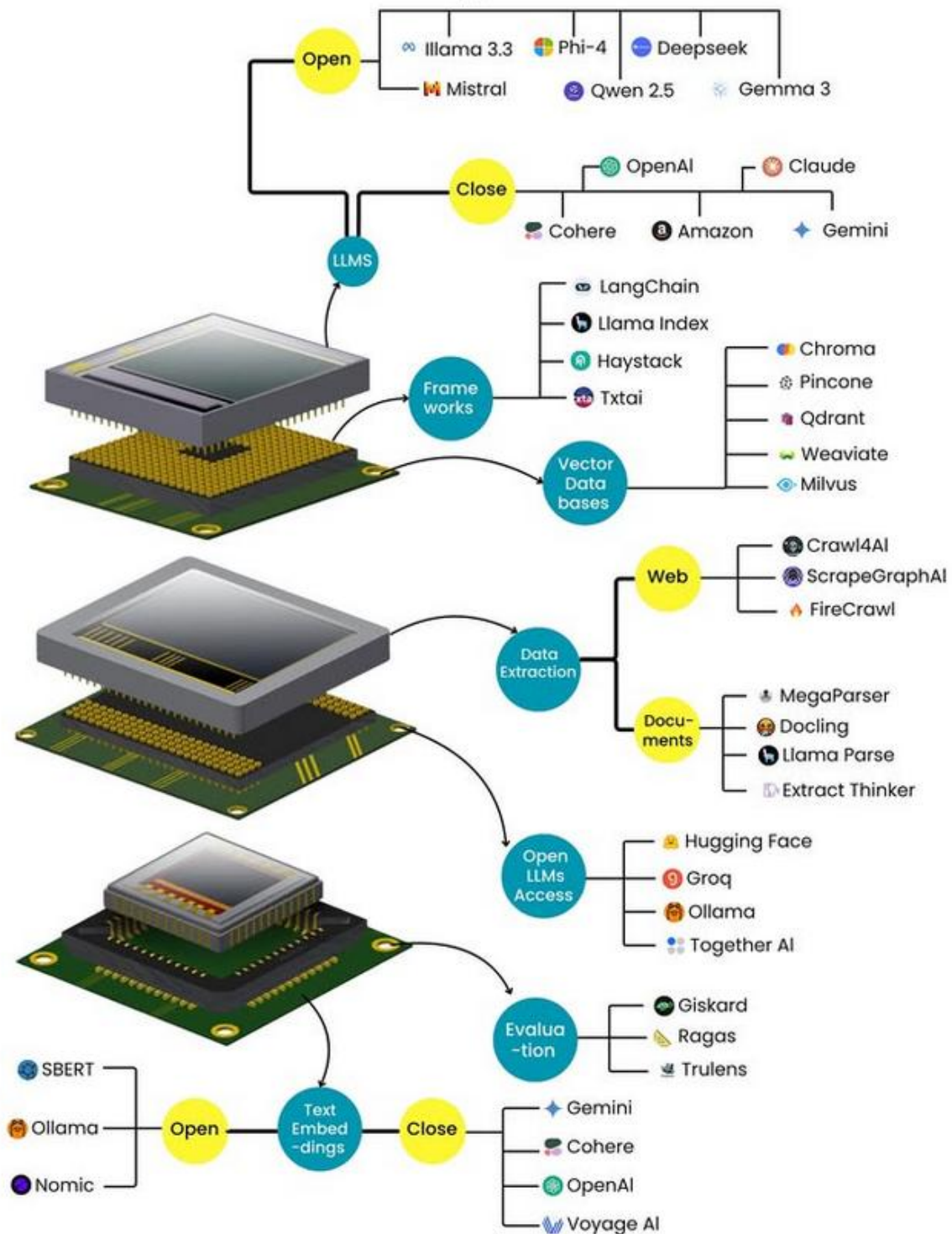
Building a Robust RAG System



4.2 RAG – Moving parts



RAG Developer's Stack



4.4 Open source RAG stack



12 Must Know RAG Terms

In Plain English

Retrieval



When the AI goes and looks up information from a library or database before answering.

Embedding



Turning words into numbers so the AI can compare meanings.

Vector Database



A giant library of embeddings the AI searches through.

Retriever



The tool that grabs the right chunks of info for the AI.

Chunking



Cutting documents into smaller parts so the AI can read them.

Context Window



The limit of how much text the AI can look at at once.

Grounding



Making sure answers are based on real facts, not guesses.

Re-Ranking



Sorting search results so the best and most relevant info comes first.

Hybrid Search



Mixing keyword search ("find this exact word") with meaning (fuzzy) search.

Agentic RAG



When RAG is combined with an agent that can reason, plan steps, and call tools — not just read and repeat.

Evaluation Metrics



Ways to measure how good answers are (like accuracy, relevance).

Latency



How long it takes the AI to answer when it has to look things up.